Name:- Samruddhi Sangram Yadav
Roll no. B560

Title:-
Implement the Continuous Bag of Words (CBOW) Model.
Stages can be:
a. Data preparation
b. Generate training data
c. Train model
d. Output

CODE:-

```
# A. Data Preparation

import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import skipgrams
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Lambda, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Sample text corpus
corpus = [
    "deep learning is very interesting",
    "natural language processing uses deep learning",
    "word embeddings represent words as vectors",
    "cbow predicts a target word from context words"
]

# Tokenize the text
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)

word_index = tokenizer.word_index
vocab_size = len(word_index) + 1

print("Vocabulary:")
print(word_index)
print("\nVocabulary Size:", vocab_size)

# Convert sentences into sequences of integers
sequences = tokenizer.texts_to_sequences(corpus)
print("\nTokenized Sequences:")
print(sequences)
```

```
# B. Generate CBOW Training Data

window_size = 2
contexts = []
targets = []

for sequence in sequences:
    for i in range(window_size, len(sequence) - window_size):
        context = (
            sequence[i - window_size:i]
            + sequence[i + 1:i + window_size + 1]
        )
        target = sequence[i]

        contexts.append(context)
        targets.append(target)

contexts = np.array(contexts, dtype="int32")
targets = np.array(targets, dtype="int32")

print("\nContext Words:")
print(contexts)

print("\nTarget Words:")
print(targets)

# Convert target words to one-hot vectors
targets_one_hot = tf.keras.utils.to_categorical(
    targets, num_classes=vocab_size
)


# C. Build and Train the CBOW Model

embedding_dim = 50

model = Sequential([
    Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim,
        input_length=window_size * 2
    ),
    Lambda(lambda x: tf.reduce_mean(x, axis=1)),
    Dense(vocab_size, activation="softmax")
```

```
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

print("\nTraining the CBOW Model...")

model.fit(
    contexts,
    targets_one_hot,
    epochs=100,
    verbose=0
)

print("Training Completed!")


# D. Output: Predict the Target Word

index_word = {index: word for word, index in word_index.items()}

sample_context = contexts[0:1]
predictions = model.predict(sample_context, verbose=0)

predicted_index = np.argmax(predictions[0])
actual_index = targets[0]

print("\n--- CBOW Output ---")
print("Context Words:",
      [index_word[i] for i in sample_context[0]])
print("Actual Target Word:", index_word[actual_index])
print("Predicted Target Word:", index_word[predicted_index])

# Display word embeddings
embedding_weights = model.layers[0].get_weights()[0]

print("\nWord Embedding for 'learning':")
print(embedding_weights[word_index["learning"]])

print("\nCBOW Practical Executed Successfully!")
```

OUTPUT:-

%runfile C:/Users/CD/.spyder-py3/test1.py –wdir
Libraries imported successfully!

Original Dataset:

Natural language processing helps computers understand human language.
Machine learning is an important part of artificial intelligence.
Deep learning uses neural networks to learn complex patterns.
Natural language processing uses machine learning techniques.
Word embeddings represent words as numerical vectors.
The CBOW model predicts a target word using surrounding context words.
Machine learning and deep learning are useful in language processing.
Word embeddings help computers understand relationships between words.

Preprocessed Sentences:
1. natural language processing helps computers understand human language
2. machine learning is an important part of artificial intelligence
3. deep learning uses neural networks to learn complex patterns
4. natural language processing uses machine learning techniques
5. word embeddings represent words as numerical vectors
6. the cbow model predicts a target word using surrounding context words
7. machine learning and deep learning are useful in language processing
8. word embeddings help computers understand relationships between words

Total number of sentences: 8
Total number of unique words: 48
Total vocabulary size including padding index: 49

Word Index:
{'learning': 1, 'language': 2, 'processing': 3, 'machine': 4, 'word': 5, 'words': 6, 'natural': 7, 'computers': 8, 'understand': 9, 'deep': 10, 'uses': 11, 'embeddings': 12, 'helps': 13, 'human': 14, 'is': 15, 'an': 16, 'important': 17, 'part': 18, 'of': 19, 'artificial': 20, 'intelligence': 21, 'neural': 22, 'networks': 23, 'to': 24, 'learn': 25, 'complex': 26, 'patterns': 27, 'techniques': 28, 'represent': 29, 'as': 30, 'numerical': 31, 'vectors': 32, 'the': 33, 'cbow': 34, 'model': 35, 'predicts': 36, 'a': 37, 'target': 38, 'using': 39, 'surrounding': 40, 'context': 41, 'and': 42, 'are': 43, 'useful': 44, 'in': 45, 'help': 46, 'relationships': 47, 'between': 48}

Context-Target Training Data Generated!
Input shape (X): (69, 4)
Output shape (y): (69, 49)

Sample Context and Target Pairs:
Context words: ['language', 'processing']

Target word: natural

Context words: ['natural', 'processing', 'helps']
Target word: language

Context words: ['natural', 'language', 'helps', 'computers']
Target word: processing

Context words: ['language', 'processing', 'computers', 'understand']
Target word: helps

Context words: ['processing', 'helps', 'understand', 'human']
Target word: computers

CBOW Model Summary:
Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lambda_1 (Lambda)	?	0 (unbuilt)
dense_7 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Training the CBOW Model...
Epoch 1/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **2s** 13ms/step - accuracy: 0.0145 - loss: 3.8898
Epoch 2/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 29ms/step - accuracy: 0.0870 - loss: 3.8717

Epoch 3/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - accuracy: 0.2754 - loss: 3.8561
Epoch 4/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - accuracy: 0.4348 - loss: 3.8415
Epoch 5/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - accuracy: 0.5507 - loss: 3.8263
Epoch 6/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 22ms/step - accuracy: 0.5362 - loss: 3.8111
Epoch 7/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.5652 - loss: 3.7953
Epoch 8/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6087 - loss: 3.7795
Epoch 9/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.5942 - loss: 3.7621
Epoch 10/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - accuracy: 0.6232 - loss: 3.7439
Epoch 11/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6087 - loss: 3.7253
Epoch 12/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6087 - loss: 3.7052
Epoch 13/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6087 - loss: 3.6844
Epoch 14/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6087 - loss: 3.6622
Epoch 15/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6232 - loss: 3.6385
Epoch 16/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6087 - loss: 3.6140
Epoch 17/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6087 - loss: 3.5877

Epoch 18/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6087 - loss: 3.5602
Epoch 19/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - accuracy: 0.5942 - loss: 3.5318
Epoch 20/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - accuracy: 0.6087 - loss: 3.5014
Epoch 21/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6087 - loss: 3.4700
Epoch 22/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6232 - loss: 3.4376
Epoch 23/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6232 - loss: 3.4033
Epoch 24/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - accuracy: 0.6232 - loss: 3.3687
Epoch 25/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - accuracy: 0.6087 - loss: 3.3321
Epoch 26/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6087 - loss: 3.2945
Epoch 27/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.6087 - loss: 3.2570
Epoch 28/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.6232 - loss: 3.2175
Epoch 29/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.6087 - loss: 3.1780
Epoch 30/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6087 - loss: 3.1358
Epoch 31/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6087 - loss: 3.0941
Epoch 32/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6087 - loss: 3.0528

Epoch 33/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.6522 - loss: 3.0100
Epoch 34/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.6522 - loss: 2.9662
Epoch 35/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6522 - loss: 2.9237
Epoch 36/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6667 - loss: 2.8806
Epoch 37/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6522 - loss: 2.8357
Epoch 38/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6522 - loss: 2.7917
Epoch 39/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6667 - loss: 2.7451
Epoch 40/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6812 - loss: 2.7003
Epoch 41/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6812 - loss: 2.6553
Epoch 42/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6812 - loss: 2.6103
Epoch 43/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.6812 - loss: 2.5651
Epoch 44/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6667 - loss: 2.5213
Epoch 45/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6812 - loss: 2.4767
Epoch 46/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.6812 - loss: 2.4311
Epoch 47/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6957 - loss: 2.3873

Epoch 48/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.7101 - loss: 2.3433
Epoch 49/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.7101 - loss: 2.3002
Epoch 50/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.7101 - loss: 2.2564
Epoch 51/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.7101 - loss: 2.2132
Epoch 52/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.7246 - loss: 2.1703
Epoch 53/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.7391 - loss: 2.1284
Epoch 54/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.7391 - loss: 2.0854
Epoch 55/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.7391 - loss: 2.0440
Epoch 56/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.7391 - loss: 2.0021
Epoch 57/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.7536 - loss: 1.9621
Epoch 58/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.7681 - loss: 1.9211
Epoch 59/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.7681 - loss: 1.8823
Epoch 60/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.7826 - loss: 1.8429
Epoch 61/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.8116 - loss: 1.8048
Epoch 62/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.8116 - loss: 1.7666

Epoch 63/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.8551 - loss: 1.7286
Epoch 64/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.8551 - loss: 1.6918
Epoch 65/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.8551 - loss: 1.6565
Epoch 66/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.8551 - loss: 1.6211
Epoch 67/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.8841 - loss: 1.5863
Epoch 68/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.8986 - loss: 1.5530
Epoch 69/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.8986 - loss: 1.5194
Epoch 70/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.8986 - loss: 1.4869
Epoch 71/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9275 - loss: 1.4553
Epoch 72/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.9275 - loss: 1.4235
Epoch 73/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9275 - loss: 1.3927
Epoch 74/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9275 - loss: 1.3622
Epoch 75/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9275 - loss: 1.3322
Epoch 76/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 1.3046
Epoch 77/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9420 - loss: 1.2751

Epoch 78/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9420 - loss: 1.2473
Epoch 79/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 1.2205
Epoch 80/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 1.1932
Epoch 81/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 1.1676
Epoch 82/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9420 - loss: 1.1417
Epoch 83/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9420 - loss: 1.1166
Epoch 84/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9420 - loss: 1.0924
Epoch 85/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9420 - loss: 1.0690
Epoch 86/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 1.0449
Epoch 87/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 1.0222
Epoch 88/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 1.0007
Epoch 89/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 0.9782
Epoch 90/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 0.9571
Epoch 91/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9420 - loss: 0.9366
Epoch 92/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9420 - loss: 0.9170

Epoch 93/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9565 - loss: 0.8968
Epoch 94/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9565 - loss: 0.8777
Epoch 95/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.9565 - loss: 0.8589
Epoch 96/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9565 - loss: 0.8404
Epoch 97/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9565 - loss: 0.8230
Epoch 98/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.9565 - loss: 0.8058
Epoch 99/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9565 - loss: 0.7887
Epoch 100/100
5/5 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.9565 - loss: 0.7723

Model Training Completed!

Model Evaluation:
Loss: 0.7599
Accuracy: 95.65 %

Word vectors saved successfully!
File location: /content/vectors.txt

Word vectors loaded successfully using Gensim!

Words similar to 'learning':
is: 0.2845
and: 0.2818
are: 0.2595
uses: 0.2498
machine: 0.2284

==========
CBOW EXPERIMENT COMPLETED SUCCESSFULLY

==========

Number of sentences: 8
Unique words: 48
Embedding dimensions: 100
Window size: 2
Training examples: 69
Final training accuracy: 95.65 %
Word vector file: /content/vectors.txt